AI ACADEMY · LEVEL 1 · GRADE 6 · WEEK 34 · STUDENT LESSON

Test Edge Cases and Fairness

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Helpful Classroom Sorter

THE CONNECTED WEEK

Probe equality. Validate inputs. Label every variation.

Extend activity tests to boundaries, missing fields and later-row paths without claiming unmeasured fairness.

Keep one habit: Hold other conditions fixed → predict the boundary → record actual coverage and gaps.

MAIN LESSON · SECTION 1

Which test could reveal the hidden assumption?

> Choose revealing boundaries and condition coverage before declaring success.

Start with a routine-success blind spot

A process can pass ordinary requests and fail exactly at equality. Test just below, at and above the boundary while holding other conditions fixed, then inspect missing inputs and untested paths separately.

Recall the limits of four passing cases

Why can four successful routine cases leave a mistake undiscovered?

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Your learning goals

- I can choose tests just below, at and above a decision **boundary**.
- I can distinguish valid no-match requests from **invalid input**.
- I can use labelled catalogue variations to examine previously untested paths.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Your learning target

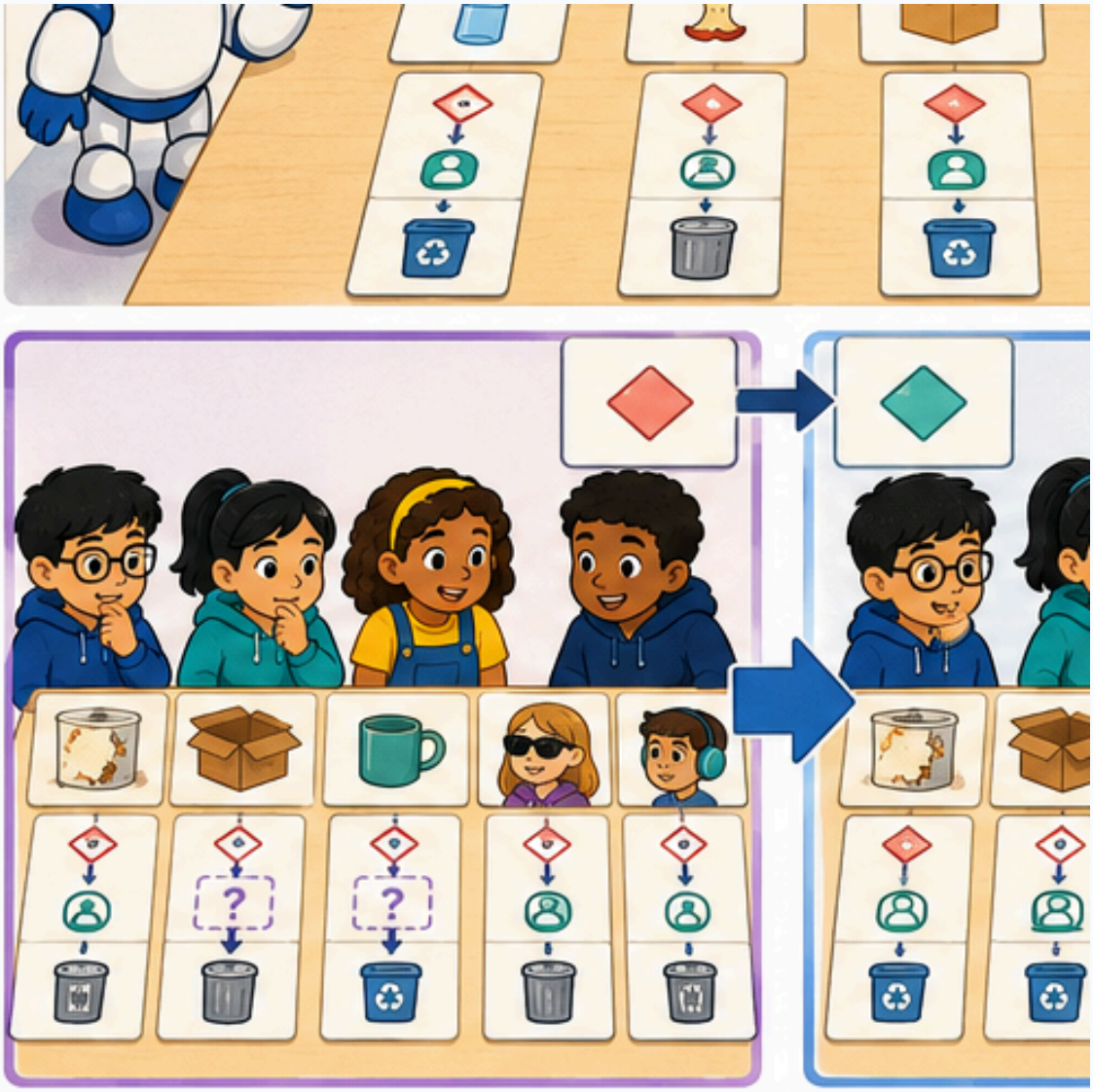
Remember before reading

Without opening the old lesson, explain one key idea from Test Normal Cases. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Design the Decision **Process**. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Humans Set Boundaries. Give an example and a mistake that the idea helps you avoid.

Unusual and **underrepresented** cases reveal **hidden assumption**s.
</details>



LEARNING TARGET Before testing, define outcomes for clean paper, metal foil, mixed packaging and an unreadable card. The team later changes mixed packaging's expected result to match its sorter.

Explain the evaluation problem.

Learner name:

Readiness for the connected explanation

This week’s end goal

By the end, I can design **boundary** and coverage tests instead of assuming routine success is enough.

MAIN LESSON · SECTION 2

Keep the activity contract fixed while expanding coverage

> Use labelled development cases, not the reserved final set.

Define revealing edge tests

Routine examples can miss a mistake at an exact limit. This week you will choose revealing cases around those limits and vary the paper catalogue to reach later successful **output**s.

Write expected results from the requirements before operating the prototype. Keep its saved instructions and each test record so a failure remains useful **evidence**.

Read the unchanged catalogue and valid inputs

Catalogue B version 1 is unchanged: B1 Bookmark drawing, 10 minutes, quiet Yes; B2 Story circle, 8 minutes, quiet No; B3 Pattern puzzle, **12 minutes**, quiet Yes; B4 Tower challenge, 15 minutes, quiet No. Search in that order.

These fictional paper values assume materials are available. **Input**s are whole minutes from 0 to 30 **inclusive** and Quiet required Yes or No. No allows quiet or nonquiet activities.

Preserve equality, both checks and all output paths

Time fits when required minutes are no more than available minutes, including equality. The quiet condition must also hold. Select the first **fitting** row with a correct reason, or report no match after every row fails.

Invalid or missing **input** requires clarification before searching. P1 validates, resets to the first row, checks both conditions, advances after failure and stops at the first fit or catalogue exhaustion.

No **output** books or publishes anything.

▼ Earlier wording, alternatives and picture transcripts

PICTURE STORY

Look Closely Before You Explain

Pictures give clues; **evidence** tells us what the clues mean.



MYSTERY A worn notebook, multilingual label, partly hidden item, and object **fitting** two bins reveal assumptions normal tests miss.

What do you notice?

Name three visible details without explaining them yet.

Circle the detail most connected to the system's decision.

Name one detail that may be decoration or distraction.

Make a first **prediction** and state the clue that supports it.

My **observation**s, **prediction**, and **evidence**:

GOOD THINKING A **prediction** can change when stronger **evidence** appears.

A closer explanation

Test below, at and above the quiet-time boundary

> Hold quiet Yes and catalogue B version 1 fixed for 9, 10 and 11 minutes.

Inspect the three adjacent requests

Three requests around ten minutes

AVAILABLE TIME	CHECK B1 TIME	RESULT FOR QUIET YES
9 minutes	10 is too long	No match in catalogue B
10 minutes	10 fits exactly	Suggest B1
11 minutes	10 fits with time left	Suggest B1

Keep Quiet required Yes and catalogue B version 1 fixed while changing available time.

Predict the strict-comparison defect

For quiet requests, 10 minutes is the first time a catalogue activity fits. Test 9, 10 and 11 with Yes so only the time changes.

A faulty comparison S version 0 uses strictly less than instead of no more than for time, while keeping the other P1 rules. It rejects B1 at exactly 10.

With quiet required, every later row also fails, so it incorrectly reports no match.

Explain why routine passes miss equality

- S can pass all four routine requests from Week 33: 11 Yes, 9 No, 6 No and 9 Yes. None has a **fitting** activity whose duration equals the request. The 10 Yes case reveals a gap those passes did not close.

  Another useful **boundary** is 8 minutes with No: equality should allow B2, while 7 No has no match. Keep both quiet values in your wider plan because No does not mean nonquiet only.

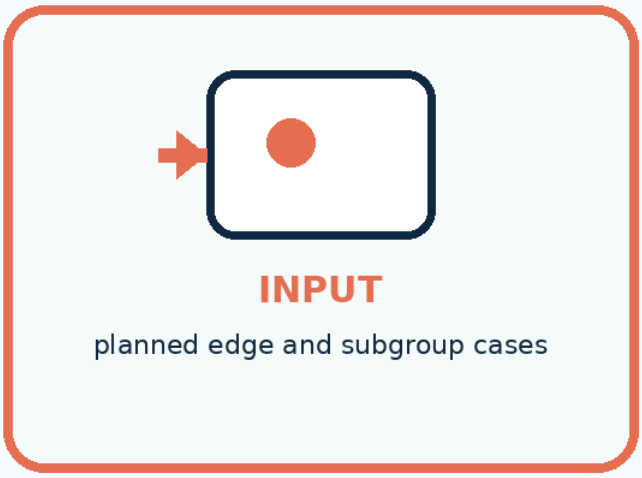
- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

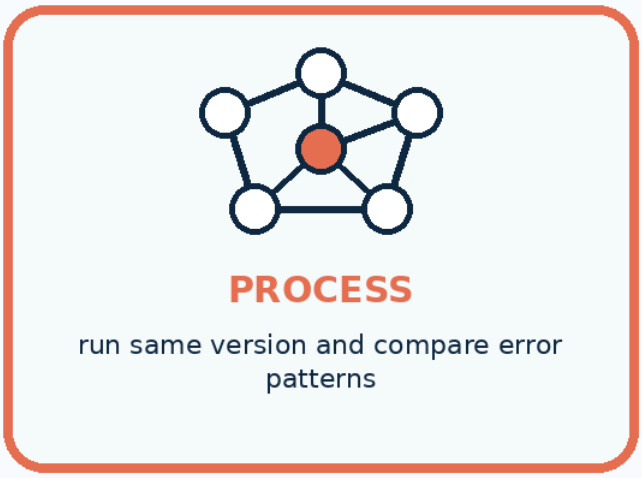
▼ Earlier wording, alternatives and picture transcripts

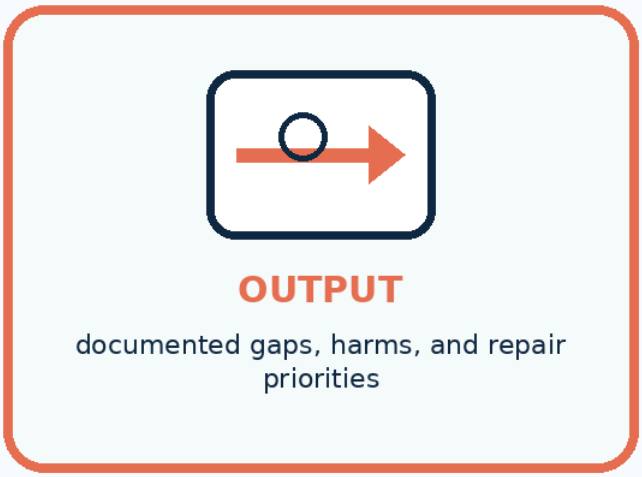
VISUAL **MECHANISM**

Input → Process → Output

Follow the information instead of imagining magic inside the machine.







INPUT planned edge and subgroup cases

PROCESS run same version and compare error patterns

OUTPUT documented gaps, harms, and repair priorities

CORE IDEA Unusual and **underrepresented** cases reveal **hidden assumption**s.

UNDER THE HOOD Edge testing explores boundaries, rare combinations, missing values, adversarial ambiguity, and subgroup conditions. Fairness analysis asks whether error burdens differ and whether fallback paths remain usable.

Draw a fresh example with all three stages:

A closer explanation

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare degraded, ambiguous and missing inputs

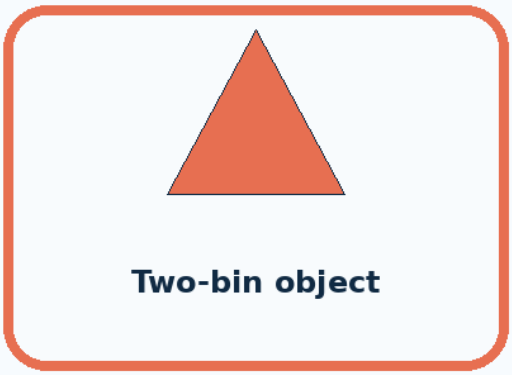
> A relevant condition needs its own evidence; one special case cannot represent everyone.

Pictures paired with their evidence



The case to investigate

CASE 1 — Worn label Clue: text partly missing. Meaning: degraded **input**.



The case to investigate

CASE 2 — Two-bin object Clue: categories overlap. Meaning: **boundary** case.



The case to investigate

CASE 3 — Different language Clue: label format changes. Meaning: access/fairness case.



The case to investigate

CASE 4 — Blank card Clue: required **input** absent. Meaning: missing-data case.

Discuss the confusing pair

- Which two pictures are easiest to confuse? What **evidence** separates them?

Keep the limit in view

BOUNDARY Edge cases are not excuses to blame users for being unusual; they are **evidence** about design limits.

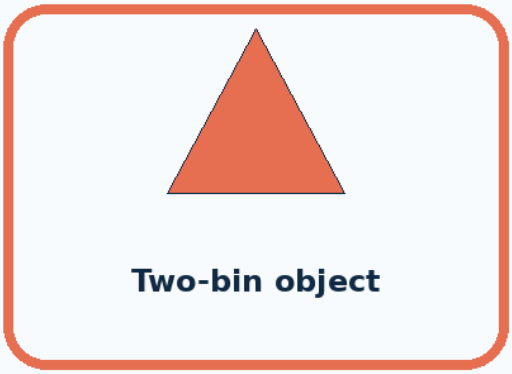
▼ Earlier wording, alternatives and picture transcripts

PICTURE GALLERY

Four Cases, Four Clues

Use the visual cue, then read the **evidence** carefully.









CASE 1 — Worn label Clue: text partly missing. Meaning: degraded **input**.

CASE 2 — Two-bin object Clue: categories overlap. Meaning: **boundary** case.

CASE 3 — Different language Clue: label format changes. Meaning: access/fairness case.

CASE 4 — Blank card Clue: required **input** absent. Meaning: missing-data case.

Change one thing in the pictured case

Change: Faulty code uses duration strictly below available time.

Prediction: Request 6 now wrongly returns none.

Reason: A's duration equals the available time and should be eligible under at most.

Trace the separate craft-book boundary

> This booklet task accepts whole minutes 0–20, distinct from the activity project’s 0–30 range.

Read the booklet catalogue and planned cases

A fictional craft-book chooser accepts requests for whole available minutes from 0 to 20.

Booklet A takes 6 minutes; B takes 12. Return first duration at most available time.

Check 5,6,7; also 0,20,21 and blank. Same catalogue applies to every user.

Return to the activity project’s ten-minute boundary

AI ACADEMY / AI-1/34 · 03 · INSPECT THE LESSON MODEL

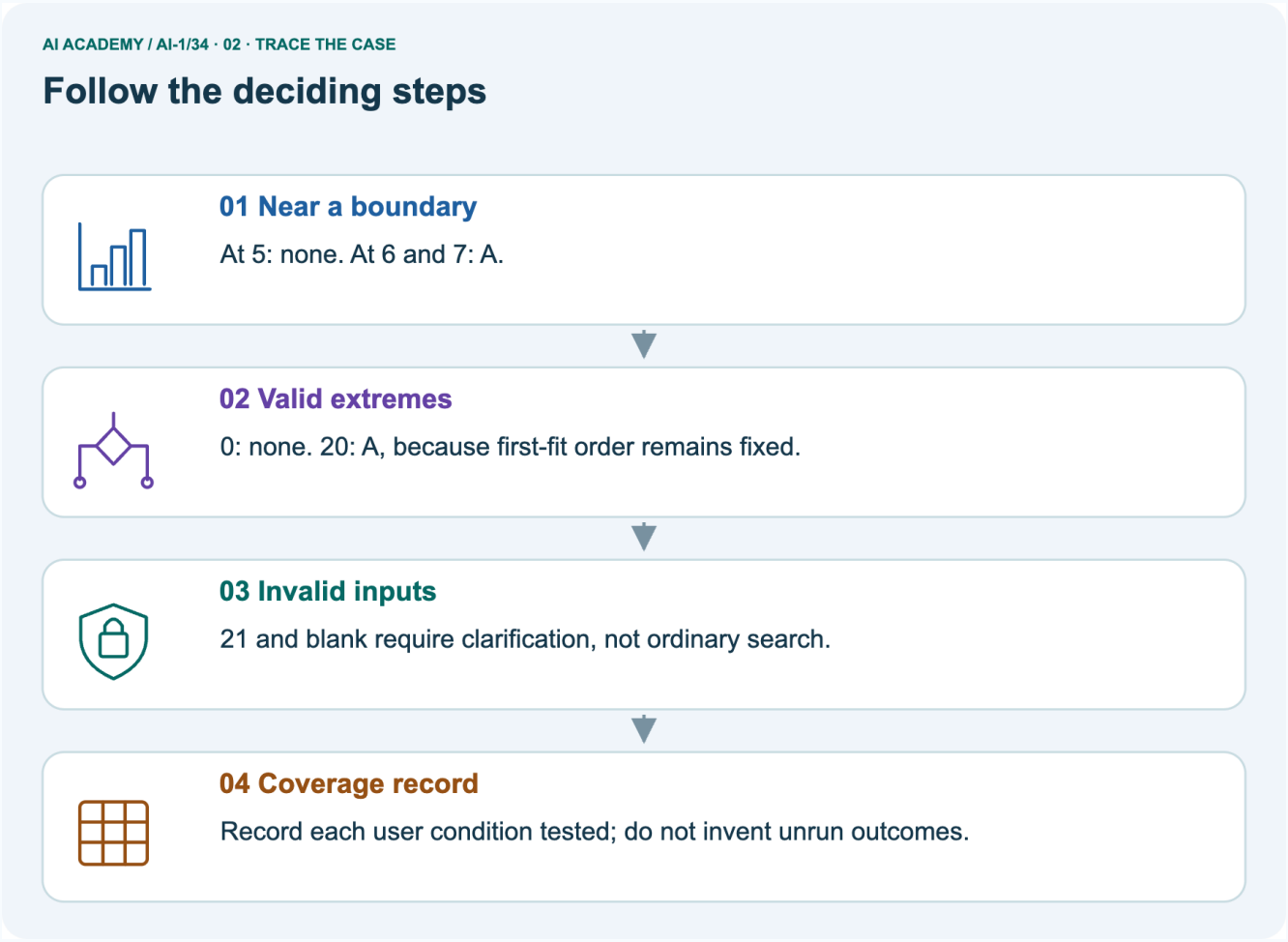
Three requests around ten minutes

Available time	Check B1 time	Result for quiet Yes
9 minutes	10 is too long	No match in catalogue B
10 minutes	10 fits exactly	Suggest B1
11 minutes	10 fits with time left	Suggest B1

Keep Quiet required Yes and catalogue B version 1 fixed while changing available time.

Follow the labelled picture with the matching explanation. Open the vector diagram

Trace equality, valid endpoints and invalid inputs



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Near a boundary	At 5: none. At 6 and 7: A.
2. Valid extremes	0: none. 20: A, because first-fit order remains fixed.
3. Invalid inputs	21 and blank require clarification, not ordinary search.
4. Coverage record	Record each user condition tested; do not invent unrun outcomes.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Identify the most revealing equality run

- Adjacent values below, at and above 6 expose an accidental strict inequality. Zero is valid even though no activity fits.

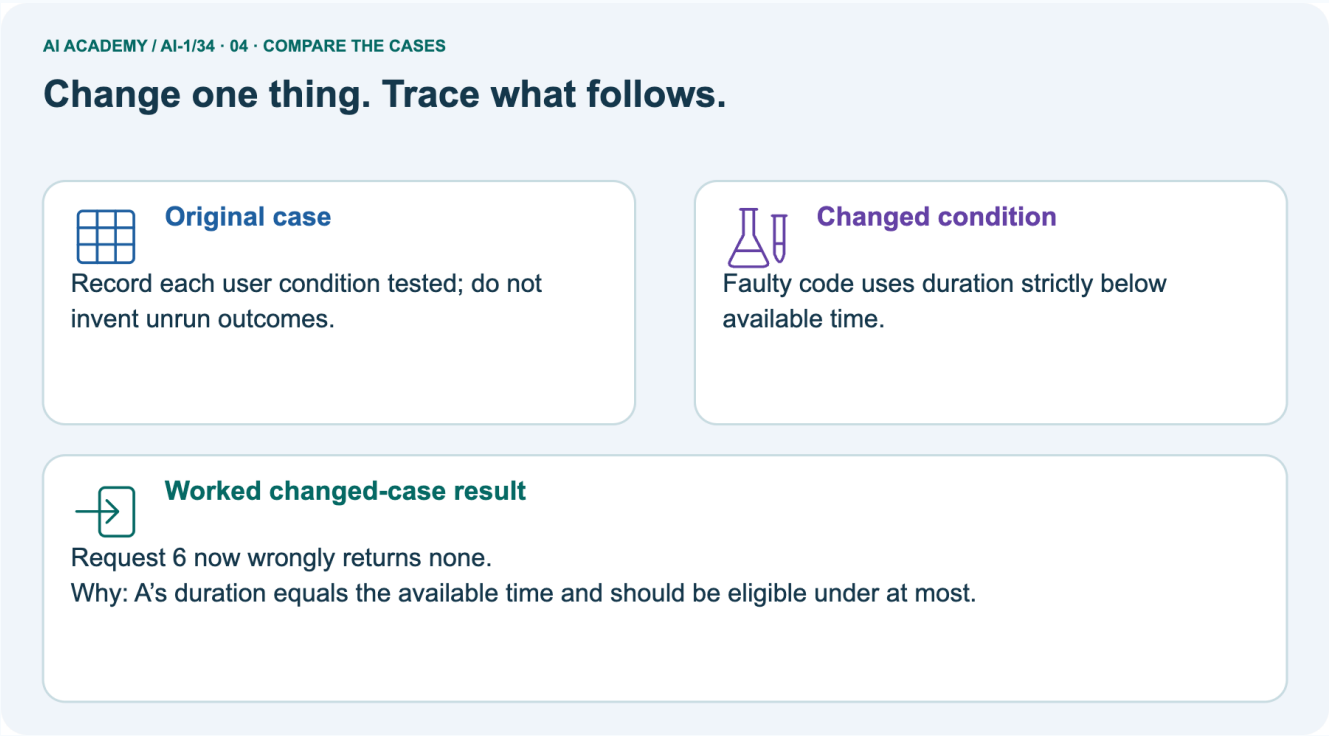
  The top valid **input** does not require selecting the longest activity: catalogue order still controls selection. Coverage also asks whose conditions were tested, which cannot be inferred from these numbers alone.

- Read the labelled picture: Which single **boundary** run most directly detects the strict-comparison bug, and why?

Predict the strict-comparison error

- Before opening the faulty-booklet picture, change at most to strictly below for the six-minute booklet. Predict the result at exactly six available minutes, and state which requirement the new result would violate.

▼ Check the changed comparison



Follow the labelled picture with the matching explanation. Open the vector diagram

Keep fairness claims within actual condition evidence

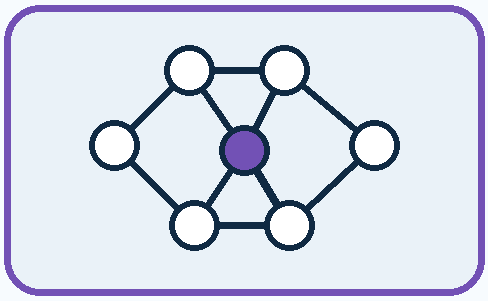
Equal **written rule**s do not prove equally good outcomes for every group; actual condition-specific **evidence** is still needed.

▼ Earlier wording, alternatives and picture transcripts

WORKED EXAMPLE

Watch the Reasoning, Not Just the Answer

Every step connects a claim to observable **evidence**.



CASE A worn notebook, multilingual label, partly hidden item, and object **fitting** two bins reveal assumptions normal tests miss.

STEP 1 — OBSERVE planned edge and subgroup cases

STEP 2 — TRACE run same version and compare error patterns

STEP 3 — CONCLUDE documented gaps, harms, and repair priorities

MODEL REASONING Use the same frozen prototype, log outcomes by condition, and compare error types. Prioritize failures with high burden or no usable fallback, then design a targeted repair without stereotyping users.

DO NOT OVERCLAIM **Edge case**s are not excuses to blame users for being unusual; they are **evidence** about design limits.

What new **evidence** could change this answer?

A closer explanation

Request using catalogue B	Expected response	Reason
0 minutes Yes	No match	Zero is valid; every activity needs more time
30 minutes Yes	B1 with a matching reason	30 is valid; B1 is the first fit

Request using catalogue B	Expected response	Reason
31 minutes Yes	Clarify minutes; no search	31 is outside the accepted range
Blank minutes with Yes	Clarify minutes; no search	The time field is missing
10.5 minutes Yes	Clarify minutes; no search	The accepted time must be a whole number
10 minutes with blank quiet	Clarify quiet; no search	Choose Yes or No before searching

The accepted range and activity durations are different kinds of **boundary**. Zero belongs to the valid range even though no activity fits. Thirty also belongs to it.

A negative time, such as minus one minute, is below zero and invalid. Do not change a blank to zero, round 10.5 to 10 or turn a blank quiet choice into No. Ask for the field to be corrected and start a new validated run.

A test record should say whether a search is expected to begin. Clarification should name the field needing correction; it should not claim the catalogue has no suitable activity.

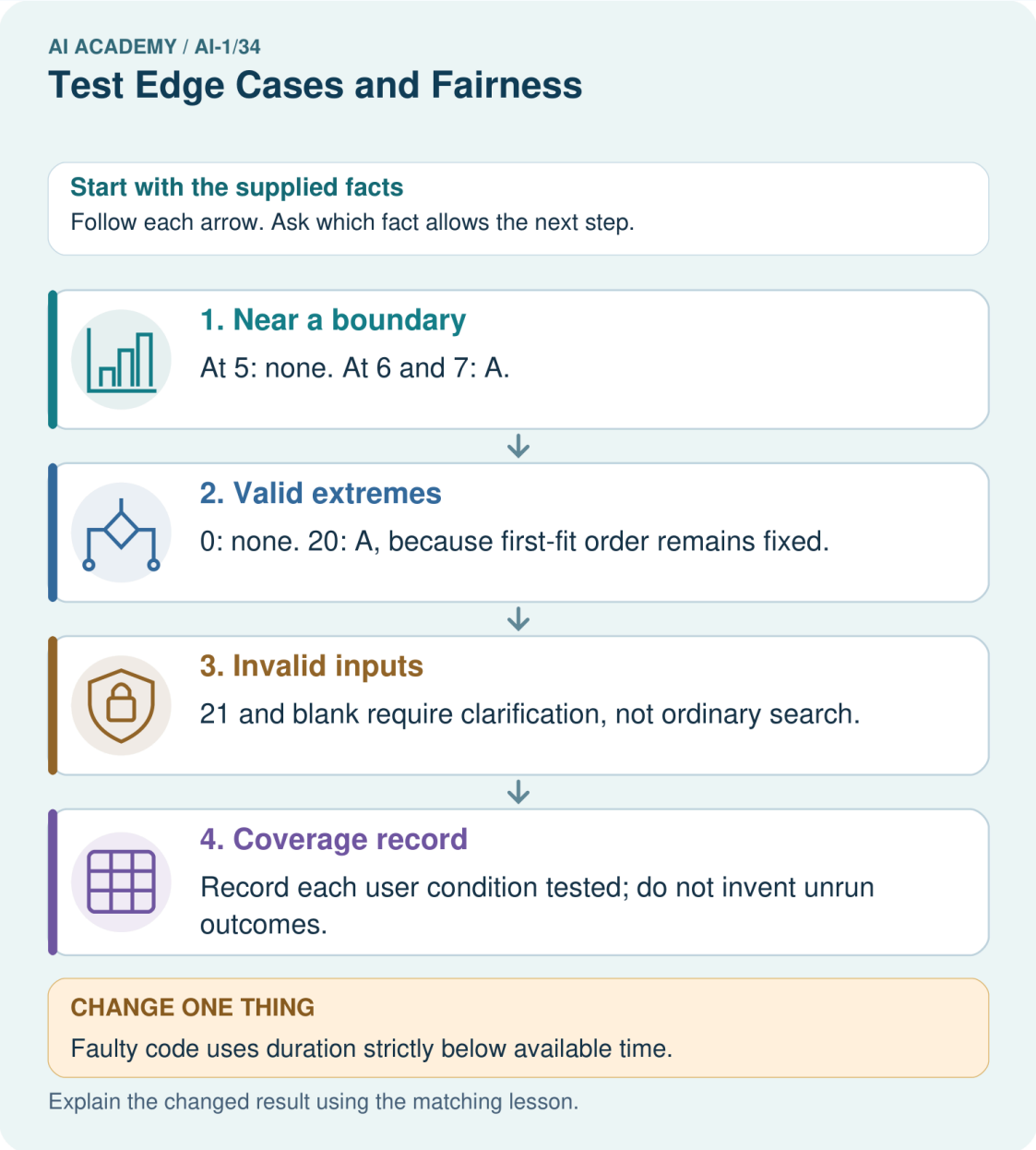
Test an unrecognised quiet value such as Maybe as well as a missing value. These check different forms of unusable **input**.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Test Edge Cases and Fairness

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Change one thing: Faulty code uses duration strictly below available time. Predict the result before checking the explanation. What remains the same?

MAIN LESSON · SECTION 7

Label every variation and preserve unrun cases as unrun

> Catalogue changes are test conditions, not silent changes to the contract.

Record purpose, catalogue, expectations and actual results

Label each case by its purpose: time equality, accepted-range endpoint, invalid field or catalogue variation. Record the exact catalogue version and request, the expected **output** and explanation, and what actually happened.

Compare the prototype with the manual baseline under identical conditions. Count every attempted case, retain failures, and distinguish cases not yet run from successful tests.

Keep development repairs separate from final evaluation

These are development tests that can guide repairs. Keep any final evaluation cases reserved by your teacher separate until the prototype version is fixed for evaluation.

Even a broad development set does not establish real-world enjoyment, material availability or speed. Next week you will repair a located defect and check whether the change breaks behaviour that worked before.

Repair both tempting claims

- CLAIM A Rare cases do not matter.
- CLAIM B One special test represents every member of a subgroup.

STRONG ANSWER FRAME I claim _____ because the **evidence** shows _____. The **process** is _____. One limit is _____.

▼ **Earlier wording, alternatives and picture transcripts**

COMPARE

Same Result, Different Reason

A comparison helps reveal the true concept **boundary**.

Case	Evidence	What it shows
Worn label	text partly missing	degraded input
Two-bin object	categories overlap	boundary case
Different language	label format changes	access/fairness case
Blank card	required input absent	missing-data case

Repair two tempting claims

Repair:

Repair:

A closer explanation

> **S passes the four routine cases but fails 10 Yes by reporting no match. Explain which comparison to inspect and why changing the expected answer to no match would be wrong. Give one additional equality case and one invalid-input case, including their expected outcomes.**

MAIN LESSON · SECTION 8

Practise: reach later rows with labelled catalogue copies

> Keep input rules, compatibility and first-match order unchanged.

Explain why a catalogue variation is needed

In B version 1, earlier B1 always fits whenever B3 or B4 could be chosen. To test a successful later-row path, make a labelled copy of the catalogue.

Keep the **input** rules, both compatibility conditions and first-match rule unchanged. Both the prototype and manual baseline must read the same test copy.

Do not overwrite the original catalogue or reuse its expected answers for the changed data.

Make B version 2 and predict a third-row success

Test copy B version 2 changes only B1's duration from 10 to 20 minutes. All other rows, quiet labels and order stay as in version 1. For 12 Yes, B1 is now too long, B2 is not quiet, and B3 fits exactly and is quiet.

The expected first suggestion is B3. This checks a third-row success and equality under a documented variation.

Make and trace B version 3 for two quiet conditions

Make test copy B version 3 with durations B1 20, B2 18, B3 17 and B4 15 minutes. Keep the original quiet labels and order. For 15 No, trace the rows and give the required output and reason. Then use 15 Yes with the same copy. Explain why the outcomes differ and what must be written on both test records.

Check validation boundaries in W2

Use workbook W2 for the connected example "Testing boundaries and varied conditions". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Repair the endpoint-is-outside claim

- Use the worked case for Testing boundaries and varied conditions. Proposed explanation: “At the limit means outside the rule”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Return to the booklet equality test

Which single **boundary** run most directly detects the strict-comparison bug, and why?

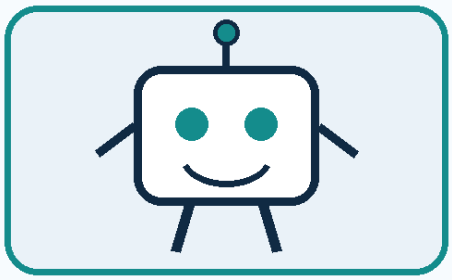
This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Earlier wording, alternatives and picture transcripts

GUIDED PRACTICE

Try It, Check It, Explain It

Use support now so you can succeed independently later.



Explain how the picture story demonstrates this idea: Unusual and **underrepresented** cases reveal **hidden assumption**s.

Use a fresh example to explain **edge case**.

Trace a new **input** → **process** → **output** example.

Find and repair the error in: One special test represents every member of a subgroup.

Invent one **edge case** and name the safe response.

PARTNER CHECK Point to the **evidence**. Do not accept an answer that only repeats the vocabulary word.

A closer explanation

Practise with less help: Check the validation boundary

Check your reasoning

Explain the picture

> **My explanation using a pictured step and the given facts**

Investigate boundary, validation and variation coverage

> Record plans separately from actual observations and subgroup evidence.

The nine development edge cases

Development edge-test pack using the activity contract: keep catalogue B version 1 for E1=(9,Yes), E2=(10,Yes), E3=(11,Yes), E4=(0,Yes), E5=(30,Yes), E6=(31,Yes), E7=(blank,Yes), E8=(10.5,Yes) and E9=(10,Maybe). Write expected outputs, reasons and whether search should start before running your saved P1 and manual baseline.

Run identical comparisons and document the variation

1. Run both methods under the same labelled conditions and retain every actual result. Then use a separate B version 2 copy changing only B1 duration to 20 for (12,Yes); do not use B version 1's expected answer for that run. Make a coverage table listing the conditions represented and those not tested. These numerical cases do not establish accessibility or fairness across real user groups: propose a relevant teacher-reviewed condition test without collecting identities or inventing results. Keep final evaluation cases unopened.

2. Keep a normal case and an **edge case** clearly separated.

3. Record what happened—even when it does not match your **prediction**.

4. Explain the result, one limitation, and the next useful test.

Retain coverage gaps and unrun tests

Case	Prediction	Observed result	What it means
Normal			
Edge			

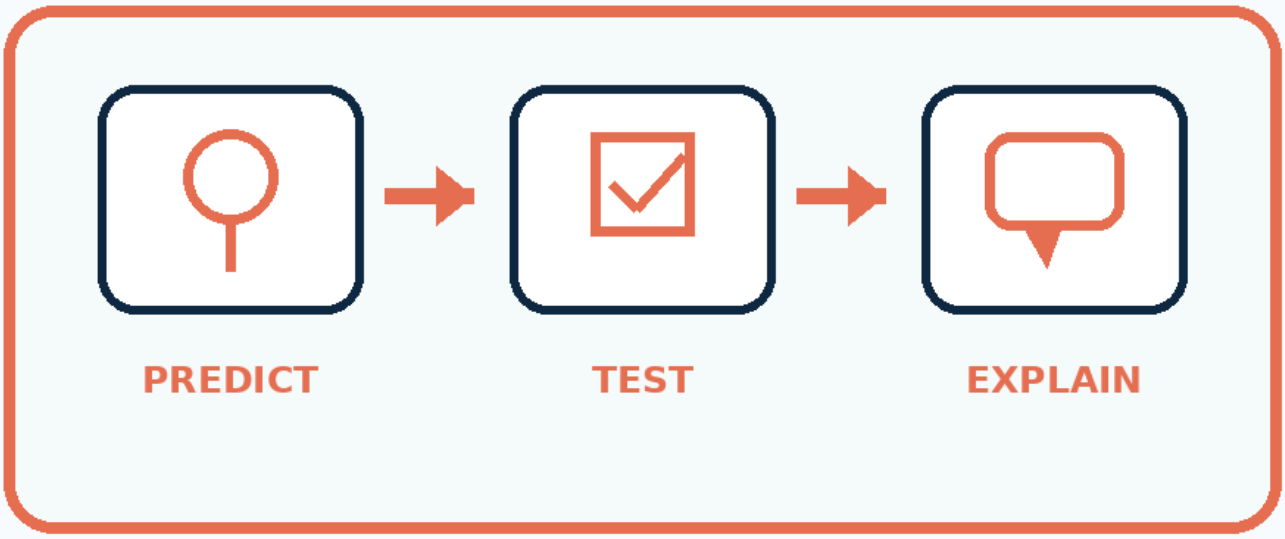
Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ **Earlier wording, alternatives and picture transcripts**

EXPLORER LAB

Edge and Fairness Test Pack

Predict first, test honestly, and explain what the **evidence** means.



State your **prediction** before the result is known.

Complete the task: Add missing, ambiguous, and **subgroup test** cards.

INDEPENDENCE After support is removed, repeat the proof with a fresh case.

Plan → try → check

Try alone: find the new capacity boundary

> Use the equipment catalogue to compare inclusive and strict capacity rules.

The independent records and requested comparisons

A new equipment selector lists E1 first, capacity 4; E2 second, capacity 8. Valid group size is a whole number from 1 to 10.

Choose the first item whose capacity is at least the group size, or no match; **invalid input** needs clarification. A faulty version uses capacity strictly greater than group size.

Independently give correct and faulty **output**s for groups 3, 4 and 5, then for group 8. Explain the defect and give the correct response to group 0. No booking occurs.

Keep your first attempt

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Find an independent capacity boundary

> **My independent explanation, trace and limits**

Week 34 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

A sorter works on clear photos. Test a blurred pencil, a dim book and an unfamiliar lunch box. What makes these useful tests?

My answer, with a trace, calculation or supporting **evidence**

> **Write or draw your reasoning and evidence.**

Create and test

Change one relevant detail. Predict the result before testing. Include a **boundary** or missing-information case when it helps reveal a limit.

My changed **input** and expected result

> **Write or draw your reasoning and evidence.**

Connect to your project

Expand the sorter test suite with edge, missing-data, and fairness cases plus expected safe behavior.

The **evidence** I will keep

> **Write or draw your reasoning and evidence.**

Transfer practice from the original programme

Before testing, define outcomes for clean paper, metal foil, mixed packaging and an unreadable card. The team later changes mixed packaging's expected result to match its sorter. Explain the evaluation problem.

> **Explain the mechanism, evidence and one limit.**

Use feedback to improve

MAIN LESSON · SECTION 11

Transfer coverage planning to the sorter portfolio

> Use each project’s own contract and permitted evidence.

Connect the contexts without mixing inputs

The main project extends the activity prototype's test coverage. Transfer the same boundary/condition method to the existing sorter portfolio here, using that sorter's own requirements. Keep activity minutes, booklet minutes and sorter input conditions separate; no second build or real pupil data collection is required.

This week's sorter test expansion

THIS WEEK'S CHECKPOINT Expand the sorter test suite with edge, missing-data, and fairness cases plus expected safe behavior.

Add sorter **boundary** cases, missing fields and relevant user conditions to the test plan before making fairness claims.

Plan relevant boundaries and review paths

- What real classroom need does this serve?
- What information is necessary—and what should stay private?
- What happens inside the **process**?
- What normal and **edge case**s will you test?
- When should it say not sure or ask a person?

Treat unusual cases as design evidence

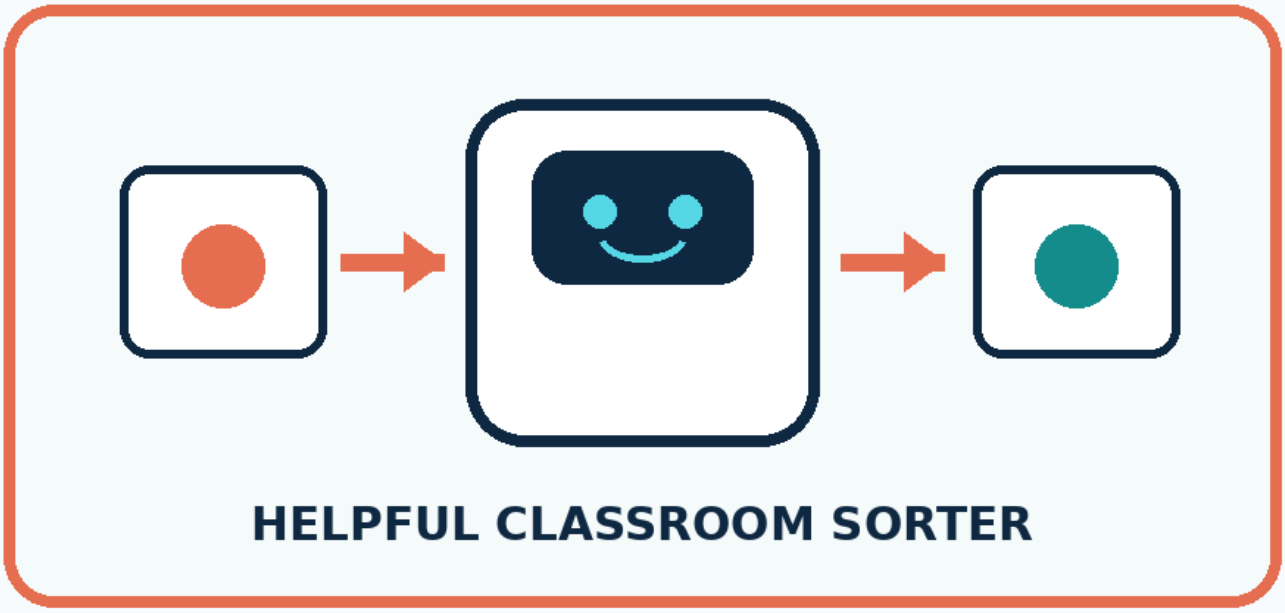
RESPONSIBLE DESIGN **Edge case**s are not excuses to blame users for being unusual; they are **evidence** about design limits.

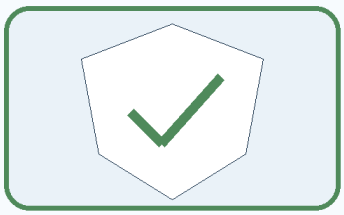
▼ **Earlier wording, alternatives and picture transcripts**

PROJECT

Helpful Classroom Sorter

Build one coherent system across the year, one **evidence**-based decision at a time.





My project decision:

Evidence for your project

MAIN LESSON · SECTION 16

Finish: explain why the boundary case matters

> Close your notes and name a coverage gap rather than claiming universal reliability.

Your exit explanation

- Close your notes. Explain this goal in your own words: Choose tests just below, at and above a decision **boundary**

My deciding step, evidence and one remaining question

Choose the next revealing check

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Your quick word guide**

REFERENCE · SECTION 5

Your quick word guide

> Use the words to explain a particular step, record or limit.

Core vocabulary

Word	Meaning	My example
edge case	unusual input near a system limit	
underrepresented	rare or absent in design evidence	
subgroup test	check for one relevant condition or user group	
hidden assumption	unstated belief revealed by failure	

Terms for the connected explanation

Word	Meaning
Boundary	A value where an input changes category or a condition changes truth
Inclusive	Including the stated endpoint or equal value
Invalid input	A value that does not meet the agreed input rules

Word	Meaning
Test variation	A documented change in the conditions used for a test
Coverage gap	A relevant condition or behaviour the current tests have not examined

Repair a claim using a word

- CLAIM TO REPAIR Rare cases do not matter.
- Use one vocabulary word and one piece of **evidence** to repair it:

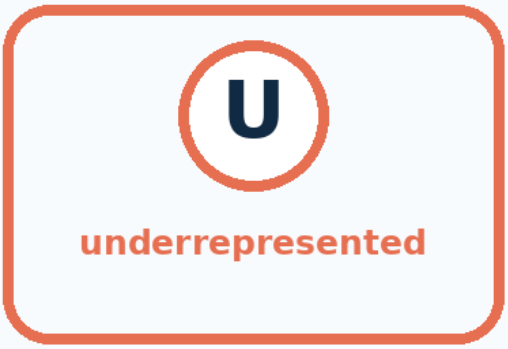
▼ Earlier wording, alternatives and picture transcripts

VOCABULARY

Words You Can See and Use

The badge is a memory cue; the definition does the real thinking work.









Misconception detective

WORD POWER A strong definition helps you decide something in a new case.

Words used in the closer explanation

▼ Choose a stronger thinking challenge

REFERENCE · SECTION 10

Choose a stronger thinking challenge

> Choose the next useful challenge rather than treating the whole ladder as compulsory.

Pick your next step

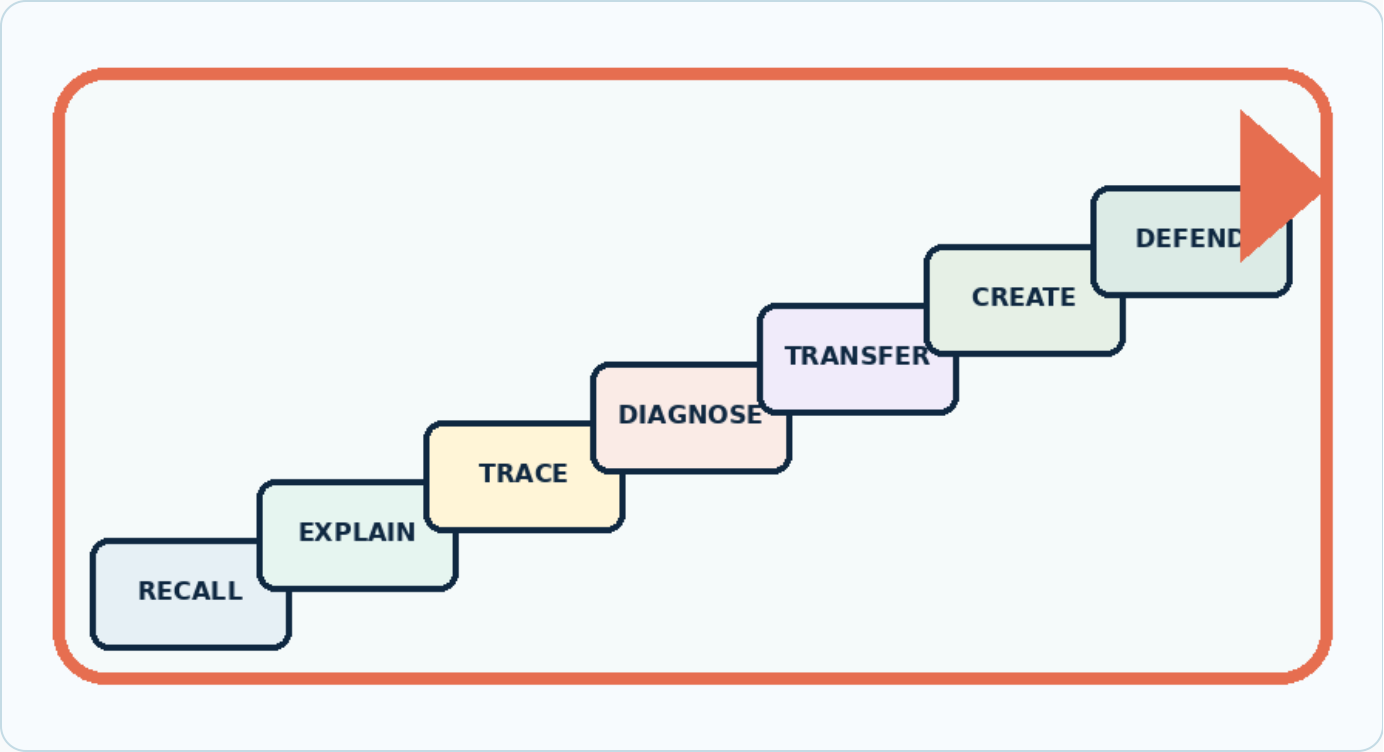
- RECALL Define **edge case**.
- EXPLAIN Unusual and **underrepresented** cases reveal **hidden assumption**s.
- TRACE Show the **input**, **process**, and **output**.
- DIAGNOSE Repair: Rare cases do not matter.
- TRANSFER Apply the idea to a new home or classroom case.
- CREATE Design one revealing test.
- DEFEND Give a claim, **evidence**, **mechanism**, limit, and safe action.

▼ Earlier wording, alternatives and picture transcripts

PRACTICE LADDER

Climb from Recall to Defense

Each step asks for a stronger kind of understanding.



RETRY RULE A coached answer becomes mastery only after you solve a different case independently.

▼ **Reference: check your explanation**

REFERENCE · SECTION 12

Reference: check your explanation

> Use after practice. Find an independent capacity boundary is the selected independent case.

Check the evidence and its limit

- ☐ Name or draw the fresh case.
- ☐ Trace **input**, **process**, and **output**.

- ☐ State the conclusion and strongest **evidence**.
- ☐ Name one limit, failure, missing clue, or fairness/privacy concern.
- ☐ Explain one safe next action or improvement.

Record support

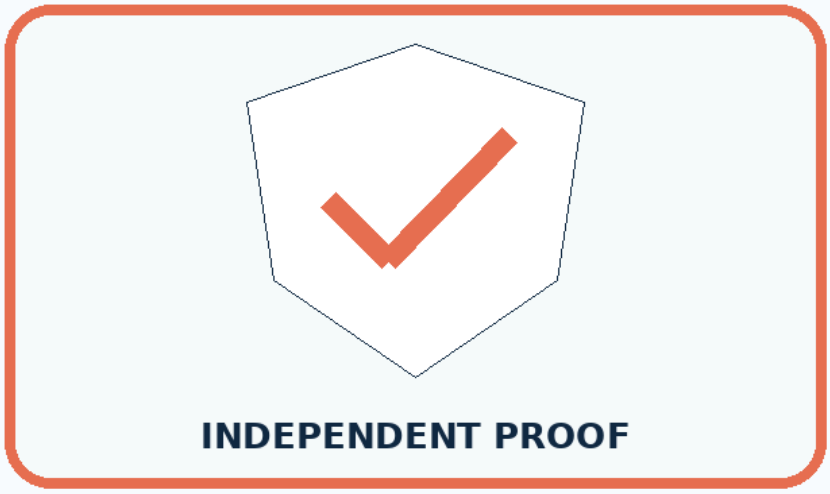
My level: ☐ Not yet ☐ With one hint ☐ Independently ☐ I can teach it

▼ Earlier wording, alternatives and picture transcripts

MASTERY

Checklist for explaining this week's idea

Use after practice. Find an independent capacity boundary is the selected independent case.



EXPLANATION CHECK Precommit expectations, distinguish validity from fitting, test equality and label every catalogue condition used.

Independent **evidence**:

SELF-CHECK Use the same frozen prototype, log outcomes by condition, and compare error types. Prioritize failures with high burden or no usable fallback, then design a targeted repair without stereotyping users.

NEXT BRIDGE Carry today's **evidence** habit into: Improve One Cause.

▼ **Extra practice: Test conditions that reveal limitations**

REFERENCE · SECTION 13

Extra practice: Test conditions that reveal limitations

> Try the question before opening its worked explanation.

Read the supplied case and question

A sorter handles clean paper but fails on crumpled paper and mixed packaging.

Design revealing tests.

Your explanation

- Which detail supports the answer, and why does it matter?

> **Write or draw your reasoning and evidence.**

▼ Check after trying

Use separately labelled clean, crumpled and mixed cases with expected **output**s or review actions declared first. Compare conditions.

Evidence or stage	Value or result
Clean paper	Routine case
Crumpled paper	Changed condition
Mixed packaging	Ambiguous material
Expected action	Declare tray or review before testing

▼ Earlier wording, alternatives and picture transcripts

Week 34 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

Test conditions that reveal limitations

A sorter handles clean paper but fails on crumpled paper and mixed packaging.

Design revealing tests.

Use separately labelled clean, crumpled and mixed cases with expected **output**s or review actions declared first. Compare conditions.

▼ Reference: original lesson checklist

REFERENCE · SECTION 15

Reference: original lesson checklist

> Use an additional checklist when it helps inspect your explanation.

Targets and evidence

By the end, I can	How I will show it
Choose tests just below, at and above a decision boundary	Explain the deciding step, show a trace or test, and state one limit.
Distinguish valid no-match requests from invalid input	Explain the deciding step, show a trace or test, and state one limit.
Use labelled catalogue variations to examine previously untested paths	Explain the deciding step, show a trace or test, and state one limit.

Keep your route and first prediction

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A fictional craft-book chooser accepts requests for whole available minutes from 0 to 20.

▼ Extra practice: pictures, repair and feedback

REFERENCE · SECTION 17

Extra practice: pictures, repair and feedback

> Attempt each question before opening its explanation. Use one browser or paper copy.

First supported question

- A fictional craft-book chooser accepts requests for whole available minutes from 0 to 20.
- Which labelled step matches this detail? Record each user condition tested; do not invent unrun outcomes.
- 1. Near a **boundary**
- 2. **Invalid input**s
- 3. Coverage record
- 4. Valid extremes
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Coverage record Record each user condition tested; do not invent unrun outcomes. Adjacent values below, at and above 6 expose an accidental strict inequality.

Zero is valid even though no activity fits. The top valid **input** does not require selecting the longest activity: catalogue order still controls selection.

Coverage also asks whose conditions were tested, which cannot be inferred from these numbers alone.

Changed-case question

- Changed-case practice: Faulty code uses duration strictly below available time. Which conclusion is supported by the supplied facts?
- 1. This one example guarantees correct results for every future **input**.
- 2. We can decide the result without examining the changed **input** or the procedure.
- 3. Request 6 now wrongly returns none.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the changed case**

After your attempt, compare with the supported explanation: Request 6 now wrongly returns none. A's duration equals the available time and should be eligible under at most.

Repair and save your work

Review Test Normal Cases (ai-1/33). Goal: Plan routine cases with expected results derived from the requirements. Short practice: Why is none correct for 8 but incorrect for 6? Use both capacity cards..

Return to this week and explain what you changed.

Review Design the Decision **Process** (ai-1/31). Goal: Design a decision **process** that covers valid, invalid and no-match requests.

Short practice: Where does the first original rejection occur, and why is K3 not originally selected?. Return to this week and explain what you changed.

Review Humans Set Boundaries (ai-1/28). Goal: Distinguish a system's capability, the correctness of its **output** and permission to take an action. Short practice: Which original proposal is authorised?

Why do the other two need separate approval?. Return to this week and explain what you changed.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

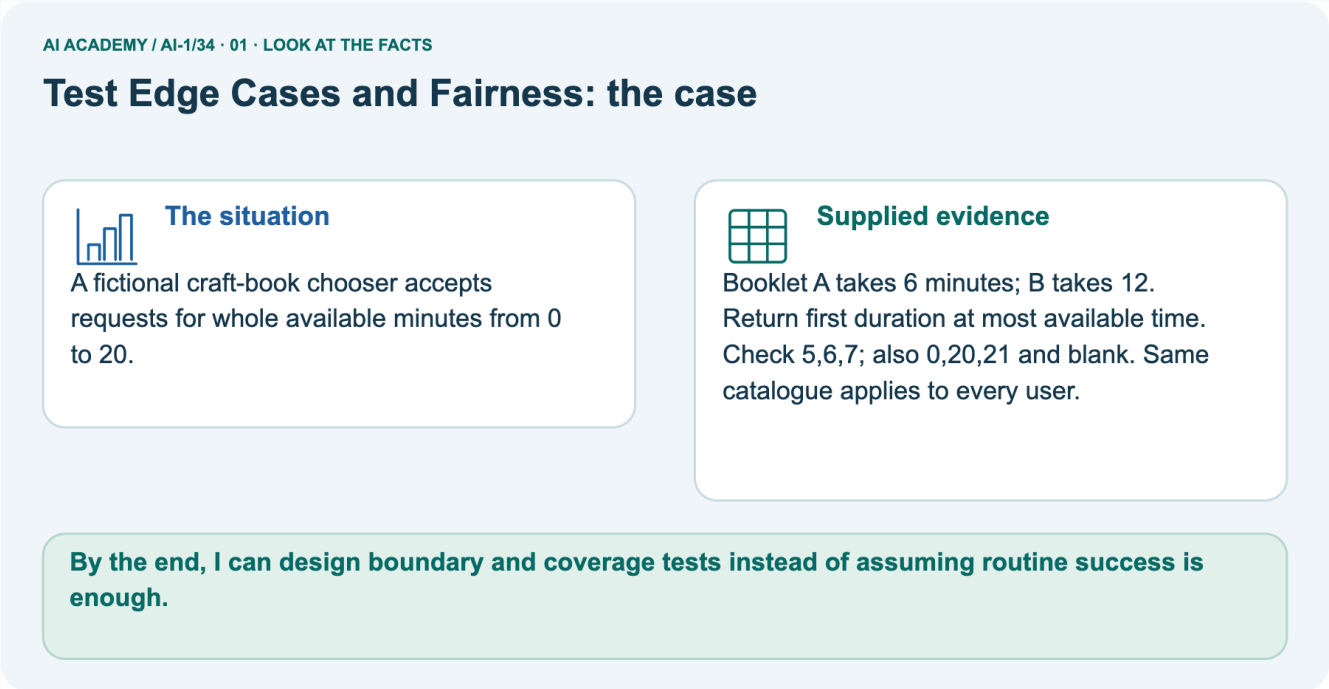
Keep your completed work

▼ Reference: the six-picture journey

Reference: the six-picture journey

> Revisit the picture that serves the current question; key diagrams already sit beside the main explanation.

01. Test Edge Cases and Fairness: the case



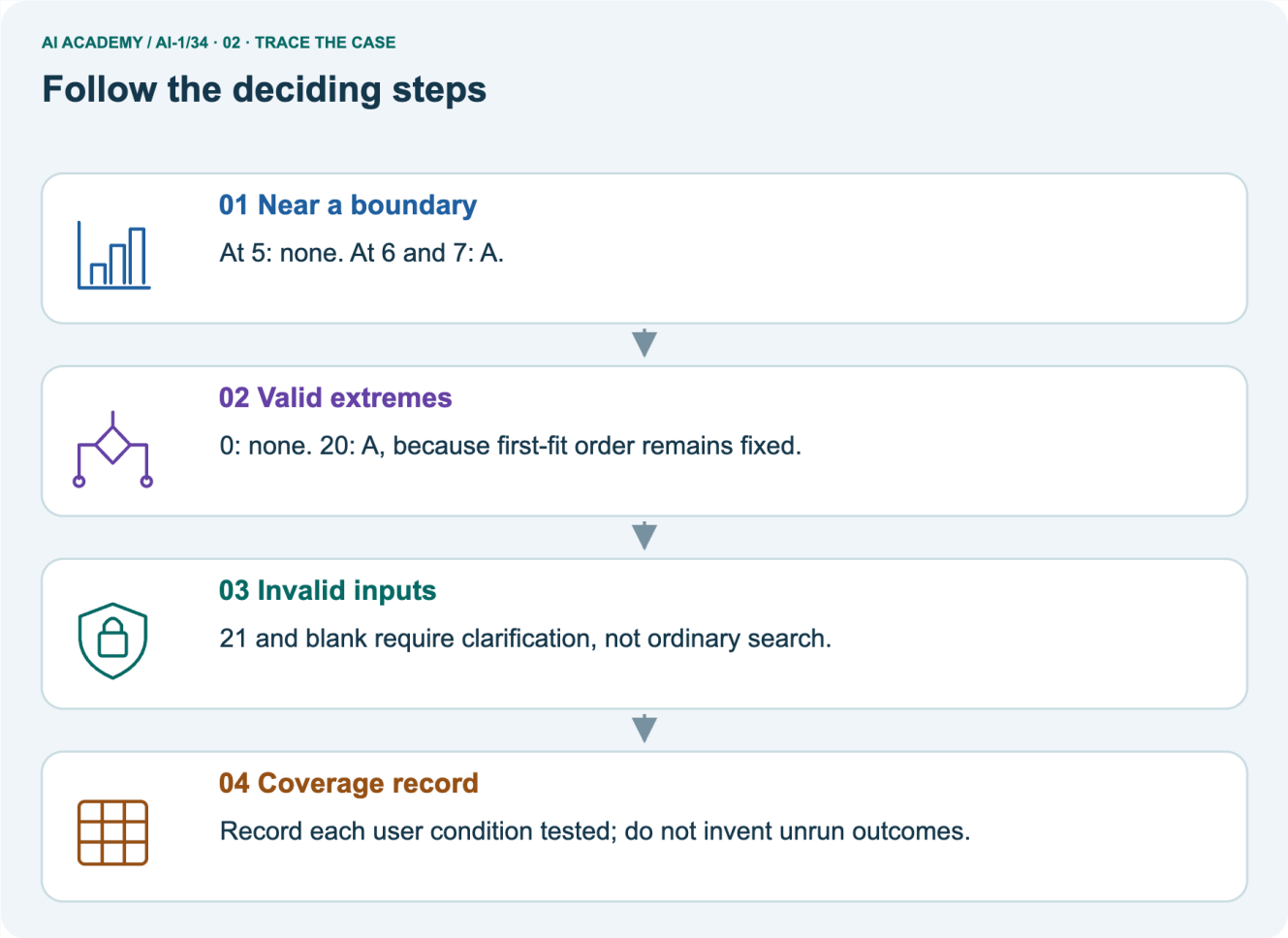
Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional craft-book chooser accepts requests for whole available minutes from 0 to 20. Supplied facts: Booklet A takes 6 minutes; B takes 12. Return first duration at most available time.

Check 5,6,7; also 0,20,21 and blank. Same catalogue applies to every user.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



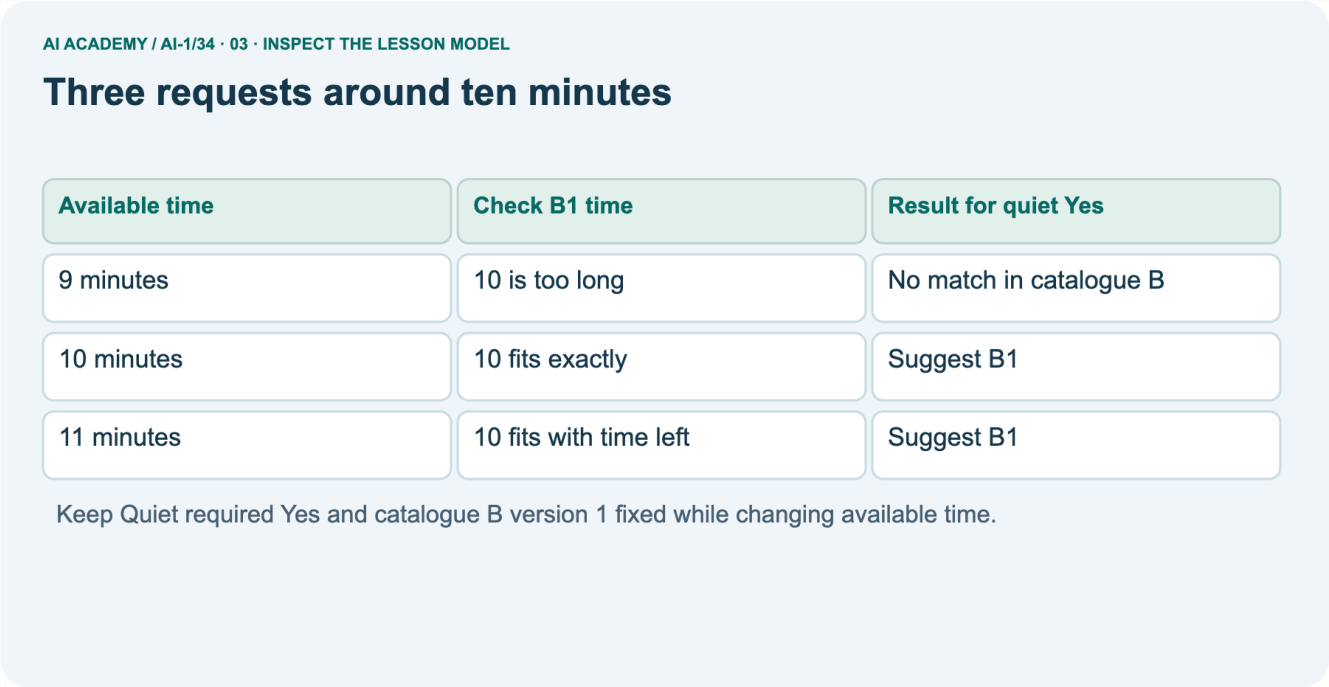
Follow the labelled picture with the matching explanation. Open the vector diagram

Near a **boundary**: At 5: none.

At 6 and 7: A. → Valid extremes: 0: none. 20: A, because first-fit order remains fixed. → **Invalid input**s: 21 and blank require clarification, not ordinary search. → Coverage record: Record each user condition tested; do not invent unrun outcomes.

Point to each numbered stage and explain which recorded **input** it uses.

03. Three requests around ten minutes

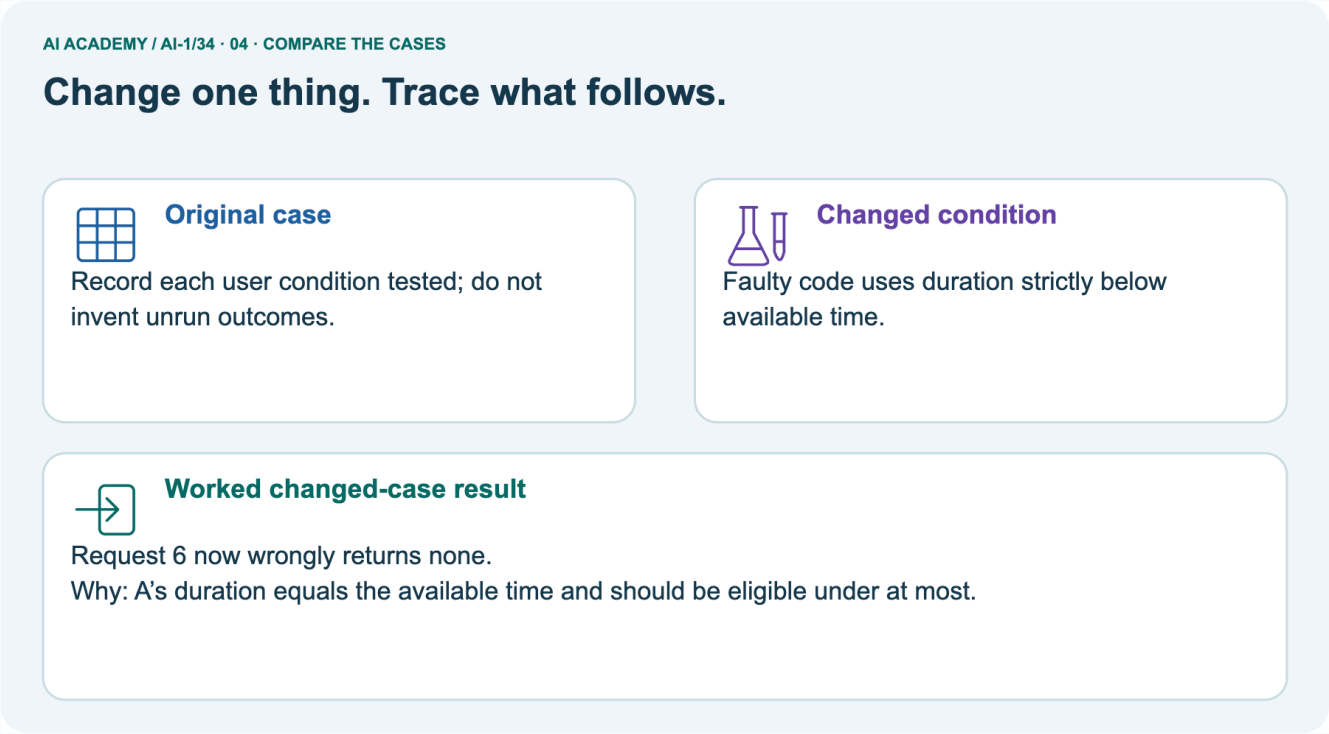


Follow the labelled picture with the matching explanation. Open the vector diagram

Three requests around ten minutes. 9 minutes | 10 is too long | No match in catalogue B. 10 minutes | 10 fits exactly | Suggest B1. 11 minutes | 10 fits with time left | Suggest B1

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



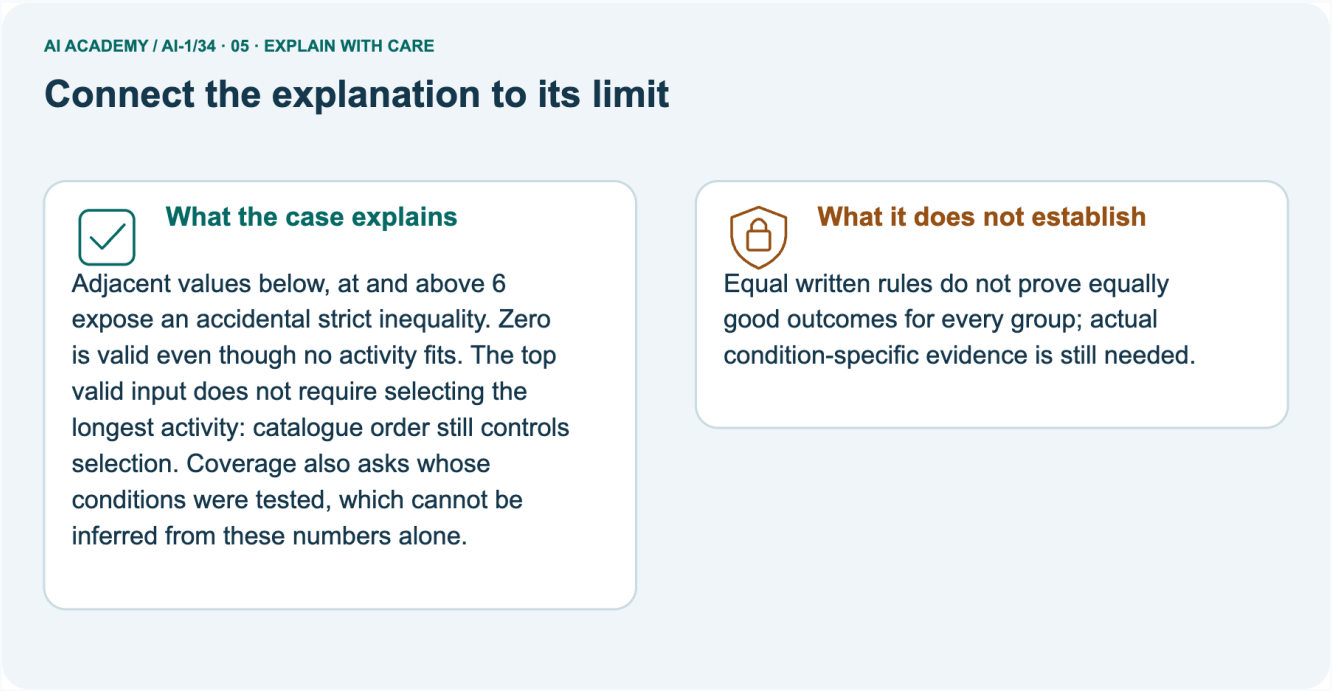
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Record each user condition tested; do not invent unrun outcomes. Changed condition: Faulty code uses duration strictly below available time. Expected result: Request 6 now wrongly returns none.

Reason: A's duration equals the available time and should be eligible under at most.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

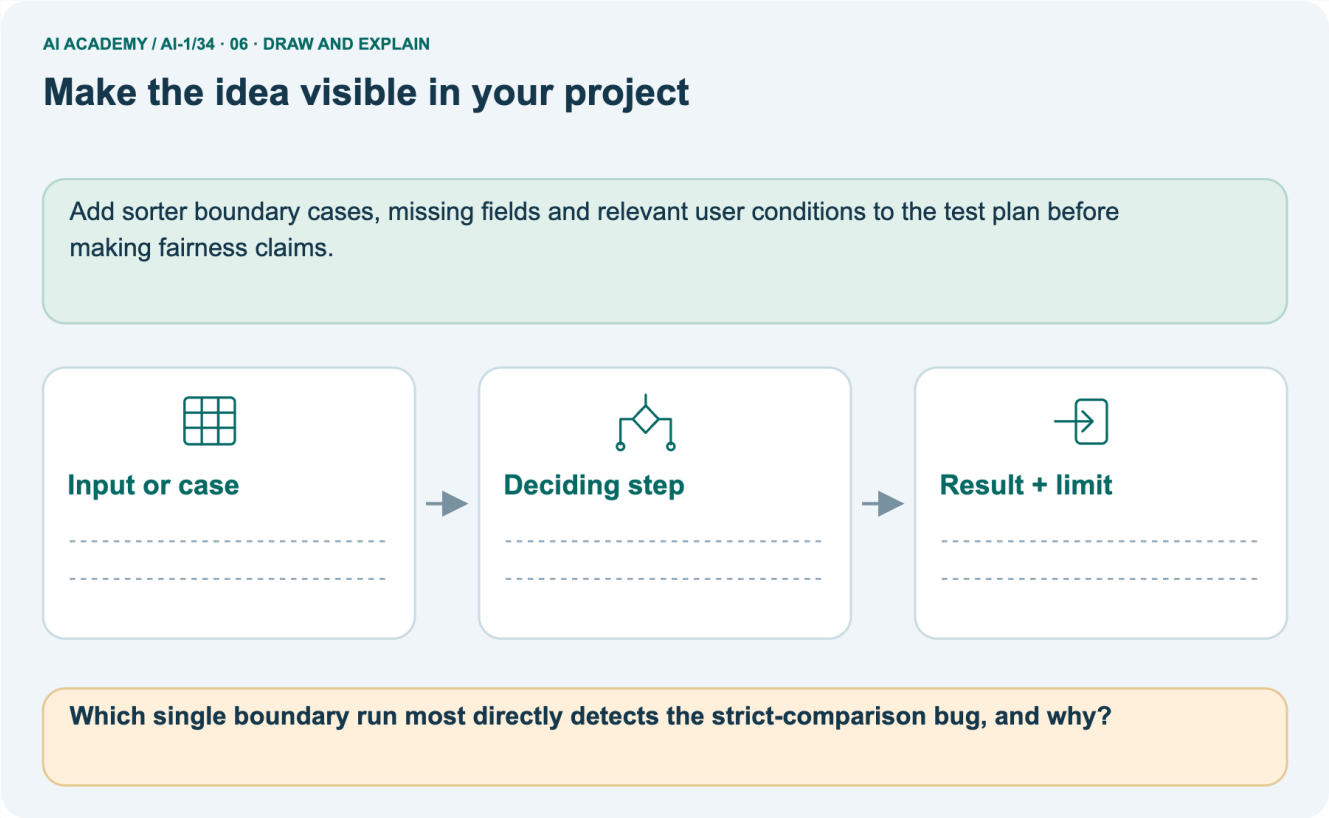
Adjacent values below, at and above 6 expose an accidental strict inequality. Zero is valid even though no activity fits.

The top valid **input** does not require selecting the longest activity: catalogue order still controls selection. Coverage also asks whose conditions were tested, which cannot be inferred from these numbers alone.

Limit: Equal **written rule**s do not prove equally good outcomes for every group; actual condition-specific **evidence** is still needed.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

Add sorter **boundary** cases, missing fields and relevant user conditions to the test plan before making fairness claims. Draw your **input**, deciding step, result and limit.

Supported question: Which single **boundary** run most directly detects the strict-comparison bug, and why?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Test Edge Cases and Fairness: the case

02. Follow the deciding steps

03. Three requests around ten minutes

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.